

GetSubRoles Class

Apex Code

```
public with sharing class GetSubRoles {
    public class Input {
        @InvocableVariable(required=true description='Role ID or fuzzy role name to search for')
        public String roleIdIdentifier;
    }

    public class Output {
        @InvocableVariable(description='Role Name')
        public String roleName;
        @InvocableVariable(description='Role ID')
        public String roleId;
        @InvocableVariable(description='Parent Role Name')
        public String parentRoleName;
        @InvocableVariable(description='Parent Role ID')
        public String parentRoleId;
        @InvocableVariable(description='List of Sub Role sObjects')
        public List<UserRole> subRoles;
        @InvocableVariable(description='Result message')
        public String message;

        public Output(
            String roleName,
            String roleId,
            String parentRoleName,
            String parentRoleId,
            List<UserRole> subRoles,
            String message
        ) {
            this.roleName = roleName;
            this.roleId = roleId;
            this.parentRoleName = parentRoleName;
            this.parentRoleId = parentRoleId;
            this.subRoles = subRoles;
            this.message = message;
        }
    }

    @InvocableMethod(label='Get Sub Roles' description='Query sub-roles by role ID or fuzzy role name')
    public static List<Output> getSubRoles(List<Input> inputList) {
        List<Output> results = new List<Output>();
        if (inputList == null || inputList.isEmpty()) {
            results.add(new Output(null, null, null, null, new List<UserRole>(), 'No input provided'));
            return results;
        }
    }
}
```

```

    }

    Input queryInput = inputList[0];
    List<UserRole> roles = RoleHelper.querySubRoles(queryInput.roleIdentifier);

    if (roles.isEmpty()) {
        results.add(new Output(null, null, null, null, new List<UserRole>(), 'No roles
found'));
        return results;
    }

    for (UserRole role : roles) {
        List<UserRole> subRoles = [
            SELECT Id, Name, ParentRoleId, ParentRole.Name
            FROM UserRole
            WHERE ParentRoleId = :role.Id
        ];
        results.add(
            new Output(
                role.Name,
                role.Id,
                role.ParentRole != null ? role.ParentRole.Name : null,
                role.ParentRoleId,
                subRoles,
                'Role retrieved successfully'
            )
        );
    }

    return results;
}
}

```

Methods

getSubRoles(inputList)

INVOCABLEMETHOD

Signature

```
public static List<Output> getSubRoles(List<Input> inputList)
```

Parameters

Name	Type	Description
inputList	List<Input>	

Return Type

List<Output>

Classes

Input Class

Fields

roleIdentifier
INVOCABLEVARIABLE

SIGNATURE

```
public roleIdentifier
```

TYPE

String

Output Class

Fields

roleName
INVOCABLEVARIABLE

SIGNATURE

```
public roleName
```

TYPE

String

roleId
INVOCABLEVARIABLE

SIGNATURE

```
public roleId
```

TYPE

String

parentRoleName
INVOCABLEVARIABLE

SIGNATURE

```
public parentRoleName
```

TYPE

String

parentRoleId

INVOCABLEVARIABLE

SIGNATURE

```
public parentRoleId
```

TYPE

String

subRoles

INVOCABLEVARIABLE

SIGNATURE

```
public subRoles
```

TYPE

List<UserRole>

message

INVOCABLEVARIABLE

SIGNATURE

```
public message
```

TYPE

String

Constructors

Output(roleName, roleId, parentRoleName, parentRoleId, subRoles, message)

SIGNATURE

```
public Output(String roleName, String roleId, String parentRoleName, String
parentRoleId, List<UserRole> subRoles, String message)
```

PARAMETERS

Name	Type	Description
roleName	String	
roleId	String	
parentRoleName	String	
parentRoleId	String	
subRoles	List<UserRole>	
message	String	